

CS285 LECTURE 6 • ZERO-TO-HERO FIELD MANUAL

Actor-Critic Architectures & Generalized Advantage Estimation (GAE)

From Scratch to Mastery: Replacing Noisy Full Rollouts with a Learned Critic and Finding the Optimal Bias-Variance Balance

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab)

Focus: The Dual Network Architecture, Temporal Difference Learning, & GAE- λ

1. What is Actor-Critic? (The Theater Director Analogy)

In Lecture 5, we learned REINFORCE: the robot runs an entire 500-step episode to the end, sums up all rewards, and updates its policy. This is called a **Monte Carlo method**. While mathematically unbiased, it is agonizingly noisy. If one small wobble happens at step 450, the entire return is ruined. Can we get immediate feedback at *every single step*? Yes! We introduce a second neural network: **The Critic**.

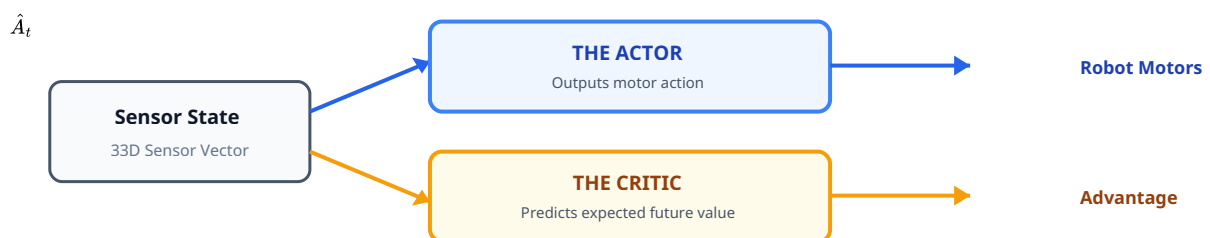
🎭 THE STAGE ACTOR & THE THEATER DIRECTOR ANALOGY

Imagine an actor rehearsing a 3-hour Shakespeare play:

- **The Monte Carlo (REINFORCE) Way:** The actor performs the entire 3-hour play in an empty theater. At midnight, after the show ends, the newspaper publishes a review: "2 out of 5 stars." The actor has no idea which specific monologue was terrible and which was brilliant. Training takes years.
- **The Actor-Critic Way:** The theater director sits in the front row during rehearsals. The instant the actor delivers a single line, the director whispers:
"That line had great emotional resonance! (+Advantage)" or *"You mumbled that phrase, speak louder! (-Advantage)"*.
 The actor doesn't wait 3 hours to learn. They get **instant, microsecond feedback on every line!**

In deep RL:

- **The Actor (π_θ):** Chooses the motor actions (a_t).
- **The Critic (V_ϕ):** Evaluates how good the resulting state is, providing immediate advantage scores to guide the Actor!



2. Temporal Difference (TD) Learning: Updating Beliefs Mid-Flight

🚗 THE ROAD TRIP TRAFFIC JAM ANALOGY

Suppose you embark on a 4-hour road trip. One hour in, you encounter a colossal traffic jam. Your GPS announces that your remaining drive is now 5 hours (total trip: 6 hours).

Do you need to wait 5 more hours until you reach your destination to conclude that your original 4-hour estimate was wrong? **Of course not!** You update your estimate **right now**. You took 1 hour of cold, hard reality + your updated estimate of the rest of the trip.

2.1 The 1-Step TD Error δ_t (Parameter Anatomy)

$$\delta_t = r(s_t, a_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Symbol	Formal Name	Plain English Meaning	Real-World Example Value
δ_t	TD Residual / Surprise Factor	How much better or worse was this step than your Critic expected?	$\delta_t = +1.45$ (Positive surprise!)
$r(s_t, a_t)$	Immediate Reality	The actual physical reward points received on this step.	+2.5 points (Sawing reward)
$\gamma V_\phi(s_{t+1})$	Discounted Future Forecast	The Critic's estimate of remaining score from the next state onward.	$0.99 \times 12.0 = 11.88$
$V_\phi(s_t)$	Prior Expectation	What the Critic thought your situation was worth <i>before</i> you took the action.	12.93 points

PLAIN ENGLISH TRANSLATION OF THE TD ERROR

"TD Error is the surprise factor: (Reality right now + What you predict for tomorrow) MINUS (What you predicted yesterday). If $\delta > 0$, you did better than expected. If $\delta < 0$, you did worse!"

3. The Bias-Variance Dilemma: Why Neither Extreme Works

When training the Actor, we face a fundamental trade-off:

- **Pure Monte Carlo (REINFORCE):** Wait until the end of the cut.
 - **Bias: ZERO.** The final score is true reality.
 - **Variance: MASSIVE.** Over 500 steps, tiny contact vibrations make the final score swing wildly between +50 and −30.
- **Pure 1-Step TD (Critic only):** Use only 1 step of reality + Critic's prediction.
 - **Variance: MINIMAL.** You only look 1 step into the future, so random noise cannot accumulate.
 - **Bias: HIGH.** Early in training, the Critic's neural network outputs nonsense guesses. If the Critic is wrong, the Actor learns bad motions.



4. Generalized Advantage Estimation (GAE- λ): The Blending Slider

John Schulman et al. (2015) invented **GAE** to smoothly interpolate between TD(0) and Monte Carlo using a single tuning parameter $\lambda \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

Parameter	Formal Name	Plain English Meaning	Recommended Setting
λ	GAE Decay Parameter	The Trust Slider: How much do you trust multi-step rollouts versus the Critic?	$\lambda = 0.95$ (Standard across all robotics)
$(\gamma \lambda)^l$	Exponential Weighting	Future TD errors δ_{t+l} are discounted exponentially so distant errors matter less.	For $l = 1$: $0.99 \times 0.95 = 0.9405$
δ_{t+l}	Future TD Residuals	The surprise factors encountered at future steps $t + 1, t + 2, \dots$	Calculated backward in time!

PLAIN ENGLISH TRANSLATION OF GAE

"GAE calculates your advantage by summing up the surprise factor today (δ_t), plus 95% of tomorrow's surprise (δ_{t+1}), plus 90% of the day after (δ_{t+2})... It blends long-term reality with short-term stability!"

5. Concrete Numerical Walkthrough: Calculating GAE by Hand

Let's calculate GAE for a 3-step sequence with $\gamma = 0.99$ and $\lambda = 0.95$. Notice that $\gamma\lambda = 0.99 \times 0.95 = \mathbf{0.9405}$.

Step t	Reward r_t	Critic $V(s_t)$	TD Error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$	GAE Advantage $\hat{A}_t$
$t = 2$	$r_2 = +5.0$	$V(s_2) = 15.0$ $V(s_3) = 16.0$	$\delta_2 = 5.0 + 0.99(16.0) - 15.0 = +\mathbf{5.84}$	$\hat{A}_2 = \delta_2 = +\mathbf{5.84}$
$t = 1$	$r_1 = +2.0$	$V(s_1) = 12.0$ $V(s_2) = 15.0$	$\delta_1 = 2.0 + 0.99(15.0) - 12.0 = +\mathbf{4.85}$	$\hat{A}_1 = \delta_1 + (\gamma\lambda)\hat{A}_2 = 4.85 + 0.9405(5.84) = +\mathbf{10.34}$
$t = 0$	$r_0 = +1.0$	$V(s_0) = 10.0$ $V(s_1) = 12.0$	$\delta_0 = 1.0 + 0.99(12.0) - 10.0 = +\mathbf{2.88}$	$\hat{A}_0 = \delta_0 + (\gamma\lambda)\hat{A}_1 = 2.88 + 0.9405(10.34) = +\mathbf{12.38}$

💡 THE BACKWARD RECURSION MAGIC

Notice how GAE is computed **backwards from the end of the episode to the beginning**:

$\text{Advantage}[t] = \text{delta}[t] + (\text{gamma} * \text{lambda}) * \text{Advantage}[t+1]$.

This simple recursive formula takes only 5 lines of PyTorch code and runs in microseconds on GPU!

6. Vectorized GAE Implementation in PyTorch

🐍 COMPLETE PYTORCH GAE FUNCTION

```
import torch

def compute_gae(rewards, values, next_values, dones, gamma=0.99, gae_lambda=0.95):
    # Compute Generalized Advantage Estimation across time steps T for B environments.
    # rewards, values, next_values, dones: Tensors of shape (T, B)
    T, B = rewards.shape
    advantages = torch.zeros_like(rewards)
    last_gae = torch.zeros(B, device=rewards.device)

    # Backward temporal recursion from T-1 down to 0
    for t in reversed(range(T)):
        non_terminal = 1.0 - dones[t].float()
        # 1-Step TD error: delta = r + gamma * V(s') * (1-done) - V(s)
        delta = rewards[t] + gamma * next_values[t] * non_terminal - values[t]
        # Recursive exponential accumulation
        last_gae = delta + gamma * gae_lambda * non_terminal * last_gae
        advantages[t] = last_gae

    # Target values for training the Critic: Returns = Advantages + Values
    returns = advantages + values
    return advantages, returns
```

7. Practitioner's Failure Modes & Debugging Checklist

Failure Mode	The Silent Bug	How to Fix It
1. Learning Rate Mismatch	If Critic learning rate equals Actor learning rate ($\alpha_c = \alpha_a$), the Critic cannot track policy changes fast enough. Its value estimates lag, feeding garbage advantages to the Actor.	Make the Critic slightly faster or larger: in SkRL, train the Critic with $\alpha_c = 1 \times 10^{-3}$ while Actor uses $\alpha_a = 3 \times 10^{-4}$.
2. Forgetting Advantage Normalization	If rewards are large (+100), advantages will be huge ($\hat{A} \sim 80$). Gradient updates will be gigantic, causing policy weights to explode.	Always normalize advantages across the batch before updating the policy: $\text{advantages} = (\text{advantages} - \text{mean}) / (\text{std} + 1\text{e-}8)$.
3. Episode Boundary Bleed	If an environment finishes at step t and resets, failing to zero out <code>last_gae</code> allows the previous episode's rewards to bleed into the new tomato!	Always multiply <code>last_gae</code> by $(1.0 - \text{done})$ at episode resets.

CS285 Lecture 6 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An